



AKSUM UNIVERSITY

FACULTY OF COMPUTING TECHNOLOGY DEPARTMENT OF

COMPUTER SCIENCE GROUP ASSIGNMENT

DESIGN AND ANALYSIS OF ALGORITHMS

TITLE: Hashing with Collision Handling: Student Record System

Group Member

NAME		ID
1) Haile	Gebremichael	1601860
2) Abel	Tikabo	1603323
3) Lwam	Tesfu	1602205

Submitted to: ekubay.

Submission date: 28/07/2018 E.C

Table of Contents

1. Introduction	1
2. Algorithm Design.....	2
2.1 Hash Function — Division Method	2
2.2 Collision Resolution — Chaining	2
2.3 Why This Algorithm?	2
Hash Table (Chaining)	3
Linear Search	3
2.4 Key Functions	3
2.5 Linear Search.....	4
3. Implementation Details.....	4
3.1 Data Structures Used	4
4. Complexity Analysis	5
4.1 Time Complexity:	5
4.1.1 Hash Table with Chaining	5
4.2 Space Complexity	5
5. Experimentation.....	6
5.1 Test Setup.....	6
6. I leaned Challenges	9
6.1 Load Factor & Table Size	9
6.2 Worst-Case Scenario	10
6.3 Rehashing.....	10
7. Conclusion.....	10
8. Reference	10
Run the Project in VS Code	11

1. Introduction

Universities manage thousands of student records and need fast access using unique identifiers such as student ID numbers. A naive approach — scanning every record one by one (linear search) — takes **$O(n)$** time, which becomes unacceptably slow as the number of students grows. For example, with 1,000 records, a linear search must check up to 1,000 entries per query. With 100,000 records, that becomes 100,000 comparisons per lookup.

A hash table solves this by mapping each student ID directly to a memory slot using a mathematical hash function, achieving **$O(1)$ average-case lookup** regardless of dataset size. However, two different IDs may map to the same slot — a situation called a **collision** — which must be handled efficiently to preserve performance.

This project implements a student information system for Aksum University using a **hash table with chaining** (an array of linked lists) for collision resolution. It also implements linear search on the same dataset as a performance baseline, and demonstrates both mathematically and experimentally how the load factor $\alpha = n/m$ directly controls hash table performance.

The real-world motivation is clear: a university registrar system that can look up any student record in near-constant time — regardless of how many students are enrolled — is far more practical than one that slows down proportionally with enrolment growth.

2. Algorithm Design

2.1 Hash Function — Division Method

The hash function maps a student ID (integer key) to a slot index in the table:

$$H(\text{key}) = \text{key} \bmod m$$

Where:

- 🚩 **key** = student ID (8–10 digit integer)
- 🚩 **m** = table size (chosen as a prime number)
- 🚩 **h(key)** = slot index in the hash table (0 to m-1)

A prime number is chosen for **m** because it reduces clustering and distributes keys more uniformly across slots. For example, if **m** were a power of 2, only the lower bits of the key would affect the hash, causing poor distribution. With a prime **m**, all bits contribute.

For this project: **m = 1009** (the next prime after 1000), **n = 1000** records.

2.2 Collision Resolution — Chaining

Each slot in the hash table holds the head pointer of a singly linked list. When two keys hash to the same slot, the new node is appended to the existing chain at that slot. This is called **separate chaining**.

Why chaining over open addressing?

- 🚩 Chaining handles any load factor ($\alpha > 1$ is possible)
- 🚩 Deletion is straightforward — just unlink the node
- 🚩 Performance degrades gracefully as α increases

2.3 Why This Algorithm?

Criterion	Hash Table (Chaining)	Linear Search	
Average search	$O(1 + \alpha) \approx O(1)$	$\Theta(n/2)$	
Worst-case search	$O(n)$ — all keys same slot	$\Theta(n)$	
Insertion	$O(1)$ average	$O(1)$ append	
Insertion	$O(1)$ average	$O(n)$ shift or mark	
Deletion	$O(m + n)$	$O(n)$	
Memory	3x–41x faster (measured)	Baseline	
Practical speed			

•

The hash table is chosen because:

- ✚ $O(1)$ average search is far superior to $O(n)$ linear search for large datasets
- ✚ Chaining handles collisions without rehashing or probing
- ✚ The division method is simple, fast, and effective for integer keys
- ✚ Prime table size minimizes clustering and improves key distribution

2.4 Key Functions

Hash Table::hash Function (int key)

Hash Table::insert (int key, Student student Data)

- Computes slot index using hash Function(key)
 - ✚ Checks for duplicate key first (updates if found)
 - ✚ If slot is empty: places node directly — no collision
 - ✚ If slot is occupied: traverses to end of chain, appends — collision counted
 - ✚ **Time:** $O(1)$ average, $O(n)$ worst case (all keys same slot)

Hash Table::search (int key)

- ✚ Computes slot index in $O(1)$
- ✚ Traverses chain at that slot comparing keys
- ✚ Returns pointer to Student data or null ptr
- ✚ **Time:** $O(1 + \alpha)$ average, $O(n)$ worst case

Hash Table::delete Record (int key)

- ✚ Finds node in chain using two-pointer technique (prev, current)
- ✚ Unlinks node and frees memory
- ✚ **Time:** $O(1 + \alpha)$ average

2.5 Linear Search::search (int key)

- ✚ Scans entire array from index 0 to count-1
- ✚ Returns pointer to Student or null ptr
- ✚ **Time:** $O(n)$ always (no shortcut possible without sorting)

Hash Table::rehash (nit new Size)

- ✚ Creates new table of given size
- ✚ Re-inserts all existing elements using new hash function
- ✚ Used when load factor becomes too high
- ✚ **Time:** $O(n + m)$ — must visit all elements and all slots

3. Implementation Details

3.1 Data Structures Used

Structure	File	Purpose	
Student struct	Student .h	Holds id, name, department, cgpa, year	
Node struct	Node .h	Linked list node: key, Student data, next pointer	
Node** table	Hash Table .h/cpp	Array of linked list heads (the hash table)	
Student* array	Linear Search .h	Unsorted array for linear search baseline	

- ✓ Table [0] -> [ID_A | data] -> [ID_B | data] -> NULL (chain of length 2)
- ✓ Table [1] -> NULL (empty slot)
- ✓ Table [2] -> [ID_C | data] -> NULL (chain of length 1)
- ✓ Table [m-1]-> [ID_D | data] -> NULL

4. Complexity Analysis

4.1 Time Complexity:

4.1.1 Hash Table with Chaining

- ✓ **Let :** n = number of elements stored
- ✓ m = table size (number of slots)
- ✓ $\alpha = n/m$ = load factor (average chain length)

Operation	Best Case	Average Case	Worst Case	
insert	$O(1)$	$O(1)$	$O(n)$	
search	$O(1)$	$\Theta(1 + \alpha)$	$O(n)$	
delete	$O(1)$	$\Theta(1 + \alpha)$	$O(n)$	
space	—	$O(m + n)$	$O(m + n)$	

Theorem (CLRS Chapter 11, Theorem 11.1 & 11.2): Under the assumption of simple uniform hashing with chaining:

- **Unsuccessful search:** $\Theta(1 + \alpha)$
 - ✚ $O(1)$ to compute hash function
 - ✚ Must traverse the entire chain at that slot (average length = α)
 - ✚ Total: $1 + \alpha$ comparisons on average
- **Successful search:** $\Theta(1 + \alpha)$
 - ✚ $O(1)$ to compute hash function
 - ✚ Traverses approximately half the chain on average
 - ✚ Total: $1 + \alpha/2 \approx \Theta(1 + \alpha)$ comparisons

Key insight: If $n = O(m)$, then $\alpha = n/m = O(1)$, so all operations are $O(1)$ average.

Numerical verification with our dataset:

- $n = 1000$, $m = 1009$ (next prime after 1000)
 - ✚ $\alpha = 1000 / 1009 = 0.9911$
 - ✚ Expected comparisons per search = $1 + \alpha = 1 + 0.9911 = 1.9911 \approx 2$
 - ✚ This is $O(1)$ — constant regardless of n .

Worst case occurs when all n keys hash to the same slot, creating a chain of length n . Search degrades to $O(n)$, identical to linear search. This is extremely unlikely with a good hash function and uniform key distribution.

4.2 Space Complexity

The hash table with chaining uses $O(m + n)$ space, where m is the table size and n is the number of records. This accounts for memory used by the table array and the linked list nodes. Linear search requires $O(n)$ space, so the hash table uses slightly more memory but offers much faster access.

Structure	Space Complexity	
Hash Table array	$O(m)$	
Linked list nodes	$O(n)$	
Hash Table total	$O(m + n)$	
Linear Search array	$O(n)$	

With $m \approx n$ (load factor ≈ 1), hash table uses $O(2n) = O(n)$ space — comparable to linear search.

Hash Function Analysis

❖ The division method $h(\text{key}) = \text{key} \bmod m$ works well when:

- ✚ m is a prime number not close to a power of 2 or 10
- ✚ Keys are uniformly distributed (8-digit random integers satisfy this)
- ✚ Keys are not multiples of m (avoided by choosing prime m)

With $m = 1009$ (prime) and $n = 1000$ random 8-digit IDs:

- ✚ Load factor $\alpha \approx 0.99$
- ✚ Expected chain length ≈ 1
- ✚ Measured collision rate $\approx 35.9\%$

5. Experimentation

5.1 Test Setup

- **Dataset:** 1,000 Ethiopian student records (self-generated)
- **Hash table size:** $m = 1009$ (next prime after 1000)
- **Load factor:** $\alpha = 0.9911$
- **Tests:** 100 searches per case
- **Timing:** C++ `chrono::high_resolution_clock` (microsecond precision)
- **Platform:** Windows, g++ -std=c++11

```

=====
||                               AKSUM UNIVERSITY ÔÇö STUDENT RECORD SYSTEM                               || |
||                               Department of CS & Software Engineering                               ||
||                               Design and Analysis of Algorithms | 2026                               ||
||                               =====                               ||
Hash Function : h(key) = key mod m (Division Method)
Collision Res.: Chaining ÔÇö Array of Linked Lists
-----

Select Data Source:
[1] Enter students manually
[2] Auto-generate 1,000 Aksum University student records
Choose (1 or 2): 2
[i] Generating 1,000 Aksum University student records...
[OK] Dataset saved to 'students.csv'
[i] Names : Ethiopian/Tigrinya (Abebe, Tigist, Haben, Kibrom, Teklay...)
[i] Fields : StudentID | Name | Department | CGPA | Year

[i] Hash Table Size (prime m) : 1009
[i] Number of records (n) : 1000

LOAD FACTOR ANALYSIS
=====
n (elements) : 1000
m (table size) : 1009
Load factor a : 0.9911
Empty slots : 9
Used slots : 1000
Max chain len : 1
Avg chain len : 1.0000
Collisions : 0
Collision rate : 0.0000%

Theoretical complexity (chaining):
Successful search : Theta(1 + 0.9911) = Theta(1.9911)
Unsuccessful search : Theta(1 + 0.9911) = Theta(1.9911)

```

```

>> LOAD FACTOR DEGRADATION EXPERIMENT ÔÇö Theta(1 + alpha)
-----
n      m      alpha      HT (us)      LS (us)      Speedup      Complexity
-----
100    53      1.887      8          14          1.750      Theta(2.887)
200    101     1.980      4          18          4.500      Theta(2.980)
400    211     1.896      5          32          6.400      Theta(2.896)
600    307     1.954      6          51          8.500      Theta(2.954)
800    401     1.995      8          83          10.375     Theta(2.995)
1000   503     1.988      13         144         11.077     Theta(2.988)
-----
[i] As n grows, HT time stays ~constant. Linear search grows O(n).

=====
||                               AKSUM UNIVERSITY ÔÇö MAIN MENU                               ||
||                               =====                               ||
[1] Search student by ID
[2] Search student by Name
[3] Insert new student record
[4] Update student record
[5] Delete student record
-----
[6] Display hash table (first 20 slots)
[7] Display all students ÔÇö sorted by CGPA
[8] Display all students ÔÇö sorted by ID
-----
[9] Statistics, analysis & collision histogram
[10] Export records / Rehash table
-----
[0] Exit
=====

```

```

Choose: 1
Enter Student ID: 160186890
[OK] Student found:
-----
ID          Name          Department          CGPA          Year
-----
160186890   Tigist Shiferaw       Mechanical Engine5ering   3.31          Year 2
-----

```

```

Choose: 2
Enter student name: Natnael Mekonnen
[OK] Student found:
-----
ID          Name          Department          CGPA          Year
-----
160186972   Natnael Mekonnen     Mathematics          2.54          Year 5
-----

```

```

Hash Table Contents (first 20 used slots)
-----
Slot[0]  ]: [160186822 | Tigist Getachew] -> NULL
Slot[1]  ]: [160186823 | Fikirte Tadesse] -> NULL
Slot[2]  ]: [160186824 | Robel Gebre] -> NULL
Slot[3]  ]: [160186825 | Selamawit Yilma] -> NULL
Slot[4]  ]: [160186826 | Rahel Mulugeta] -> NULL
Slot[5]  ]: [160186827 | Mikias Girma] -> NULL
Slot[6]  ]: [160186828 | Natnael Alemu] -> NULL
Slot[7]  ]: [160186829 | Selamawit Haile] -> NULL
Slot[8]  ]: [160186830 | Liya Lemma] -> NULL
Slot[9]  ]: [160186831 | Tsion Abate] -> NULL
Slot[10] ]: [160186832 | Liya Lemma] -> NULL
Slot[11] ]: [160186833 | Henok Getachew] -> NULL
Slot[12] ]: [160186834 | Fikirte Demeke] -> NULL
Slot[13] ]: [160186835 | Dawit Tesfaye] -> NULL
Slot[14] ]: [160186836 | Rahel Mulugeta] -> NULL
Slot[15] ]: [160186837 | Rahel Fekadu] -> NULL
Slot[16] ]: [160186838 | Mikias Wolde] -> NULL
Slot[17] ]: [160186839 | Fikirte Desta] -> NULL
Slot[18] ]: [160186840 | Selam Abate] -> NULL
Slot[19] ]: [160186841 | Robel Teshome] -> NULL
-----
Total records in table: 1000

```

From the above as sample of this but all is work so the the additional are work so examples

```

-----
>> MATHEMATICAL ANALYSIS  Θ  Theta(1 + alpha)
-----
Hash Function   : h(key) = key mod m  (Division Method)
Collision Res.  : Chaining (array of linked lists)
n = 1000 elements, m = 1009 slots
Load factor     : alpha = 1000/1009 = 0.9911

THEOREM (CLRS Ch.11  Θ  Simple Uniform Hashing + Chaining):
-----
Unsuccessful search : Theta(1 + alpha)
    O(1) hash compute + traverse full chain (avg len = alpha)

Successful search   : Theta(1 + alpha)
    O(1) hash compute + traverse ~half chain on average

With alpha = 0.9911:
    Expected comparisons = 1 + 0.9911 = 1.9911

If n = O(m) => alpha = O(1) => all ops are O(1) avg
Linear search is always Theta(n) regardless of input.

```

```

Best Case  Θ  keys at front of chain
-----
Structure      Time(us)    Found    Avg/search
-----
Hash Table      4           100      0.0400 us
Linear Search   26          100      0.2600 us
-----
Speedup: 6.5x faster

Average Case  Θ  random existing keys
-----
Structure      Time(us)    Found    Avg/search
-----
Hash Table      11           100      0.1100 us
Linear Search   224          100      2.2400 us
-----
Speedup: 20.4x faster

Worst Case  Θ  keys NOT in table
-----
Structure      Time(us)    Found    Avg/search
-----
Hash Table      10            0        0.1000 us
Linear Search   546            0        5.4600 us
-----
Speedup: 54.6x faster

True Worst Case  Θ  All 100 keys forced to slot 0
-----
Chain length = 100 (all keys in one slot)
Hash Table      : 34 us | found: 100
Linear Search   : 25 us | found: 100
[!] HT degrades to O(n) when all keys collide  Θ  chain = n

```

6. I leaned Challenges

Implementing collision handling with linked lists increased program complexity. Manual memory management in C++ required careful handling to avoid memory leaks or pointer errors.

6.1 Load Factor & Table Size

Choosing an appropriate table size and maintaining a good load factor were critical for efficiency. A poor choice could increase collisions and reduce performance.

6.2 Worst-Case Scenario

If many keys hash to the same slot, search and insertion can degrade to **$O(n)$** , similar to linear search.

6.3 Rehashing

When the table becomes too full, rehashing is needed to maintain efficiency. This process is costly as all elements must be reinserted into the new table.

7. Conclusion

In this project demonstrates that hash tables provide a highly efficient solution for managing student records in university administration systems. By reducing the time complexity of search operations from $O(n)$ to near $O(1)$, the system achieves significant performance improvements, particularly for large datasets. The use of separate chaining effectively handles collisions while maintaining data integrity and scalability.

The project also highlights the importance of careful design choices, such as selecting a prime number for the table size and maintaining an appropriate load factor to minimize collisions. While the implementation presented challenges, particularly in terms of manual memory management and pointer manipulation, it provided valuable insights into low-level programming and data structure design. Overall, the system successfully achieves its objective of delivering fast, reliable, and scalable student record management.

NB: all the 10 choose is perfectly work if you want check it.

8. Reference

Stroustrup, B. (2013). *The C++ Programming Language* (4th ed.). Addison-Wesley. This reference was used for understanding C++11 features, memory management, and implementation of data structures using pointers.

ISO/IEC. (2011). *ISO/IEC 14882:2011 – Programming Languages – C++*. This standard documentation guided the use of C++11 syntax and programming practices followed in the implementation.

Weiss, M. A. (2014). *Data Structures and Algorithm Analysis in C++* (4th ed.). Pearson. This source was useful for understanding hash table implementations, collision handling strategies, and complexity analysis.

Sedgewick, R., & Wayne, K. (2011). *Algorithms* (4th ed.). Addison-Wesley. This book provided additional insights into hashing performance, practical considerations, and algorithm efficiency.

Self-Generated Dataset. (2026). *Ethiopian Student Records Dataset (1,000 entries)*. The dataset used in this project was manually generated to simulate a realistic university student database.

Run the Project in VS Code

Steps to run the project in Visual Studio Code:

1. Install Visual Studio Code and a C++ compiler (GCC).
2. Open the project folder in VS Code.
3. Open the terminal from the menu (Terminal → New Terminal).
4. Compile the program using the command below.
5. Run the generated executable to start the program.

Compile by this

```
g++ main.cpp HashTable.cpp LinearSearch.cpp DataGenerator.cpp -o studentSystem
```

run by

```
./studentSystem
```